คู่มือหลักการทำงานของระบบตรวจจับมัลแวร์

ระบบถูกแบ่งการทำงานออกเป็น 5 เมนูหลักที่รวบรวมไว้ในไฟล์ main.py ซึ่งเปรียบเสมือนศูนย์ควบคุมของโปรแกรม

ขั้นตอนที่ 1: การรวบรวมข้อมูล (Data Gathering)

เป้าหมายของขั้นตอนนี้คือการนำไฟล์ปกติและไฟล์มัลแวร์มาสกัดเอา "ลักษณะเด่น" (Features) ออกมาเป็นตัวเลขเพื่อให้คอมพิวเตอร์เข้าใจได้

- **การเลือกโฟลเดอร์:** โปรแกรม gathering.py จะเปิดหน้าต่างให้ผู้ใช้เลือกโฟลเดอร์และกำหนดความเสี่ยง (0 = ปกติ, 1 = มัลแวร์)
- **การทำงานแบบคู่ขนาน:** โปรแกรมจะใช้ทรัพยากร CPU หลายแกนพร้อมกันเพื่อสแกนไฟล์จำนวนมากได้อย่างรวดเร็ว
- **การสกัดข้อมูล:** ไฟล์ Features_gathering.py จะทำหน้าที่อ่านข้อมูลเชิงลึกของไฟล์ เช่น ขนาดไฟล์ (file_size_bytes), ค่าความไร้ระเบียบของข้อมูล (file_entropy), และประเภทของหัวไฟล์ (magic_bytes)
- **การบันทึกผล:** ข้อมูลที่สกัดได้ทั้งหมดจะถูกเก็บลงฐานข้อมูลชื่อ malware_analytics.db

ขั้นตอนที่ 2: การจัดการฐานข้อมูล (Database Management)

ไฟล์ management.py เป็นเครื่องมือดูแลรักษาข้อมูลที่เก็บมาได้

- **การรวมข้อมูล (Merge):** สามารถนำฐานข้อมูลหลายๆ ไฟล์มารวมกันได้ โดยระบบจะกรองข้อมูลที่มีรหัสแฮช (SHA256) ซ้ำกันออกให้อัตโนมัติ
- **การล้างไฟล์ขยะ (Clean):** ระบบจะช่วยสแกนหาไฟล์ที่มีนามสกุลไม่เกี่ยวข้องหรือไม่ต้องการออกจากฐานข้อมูล เพื่อประหยัดพื้นที่และลดความสับสนของข้อมูล

ขั้นตอนที่ 3: การทำความสะอาดข้อมูลก่อนสอน AI (Data Cleaning)

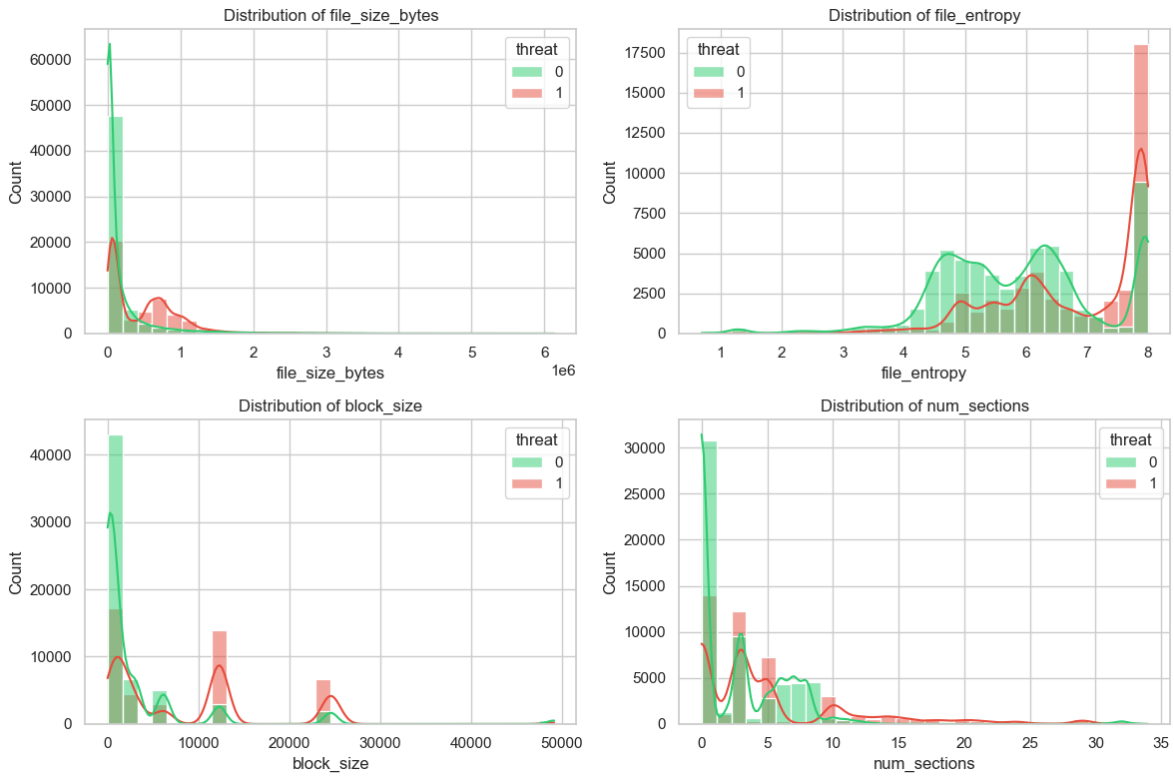
ก่อนที่ AI จะเรียนรู้ ข้อมูลจะต้องบริสุทธิ์ที่สุด ซึ่งไฟล์ cleaning.py จะรับหน้าที่นี้

- **การคัดกรองขนาดและคุณสมบัติ:** ระบบจะตัดไฟล์ที่มีขนาดใหญ่เกินไป หรือมีค่าเอนโทรปีที่ไม่อยู่ในเกณฑ์ที่กำหนดออก
- **การกำจัดข้อมูลผิดปกติ (Outlier Detection):** มีการใช้อัลกอริทึม IsolationForest เพื่อตรวจหาและตัดข้อมูลที่แปลกประหลาดมากๆ ทิ้งไป ซึ่งจะช่วยให้ AI ไม่เกิดความสับสนเวลาเรียนรู้
- **การบันทึกข้อมูลพร้อมใช้:** ข้อมูลที่สะอาดแล้วจะถูกบันทึกลงตารางใหม่ชื่อ malware_features_train_clean

ขั้นตอนที่ 4: การวิเคราะห์และทำความเข้าใจข้อมูล (EDA)

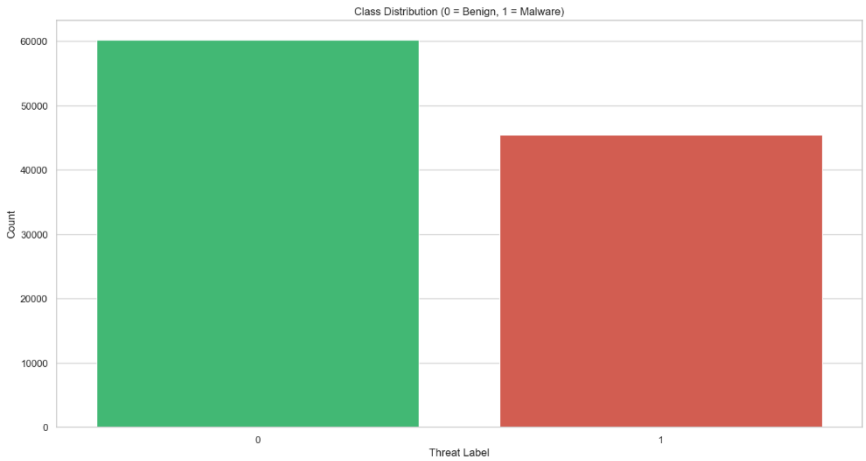
เพื่อทำความเข้าใจลักษณะของข้อมูล ค้นหารูปแบบ ตรวจสอบสิ่งผิดปกติ และดูความสัมพันธ์ระหว่างตัวแปร

กราฟความหนาแน่นแสดงการกระจายตัวของข้อมูล 4 ตัวแปร ได้แก่ การกระจายตัวของขนาดไฟล์ การกระจายตัวค่าเอนโทรปีของแซนนอน การกระจายตัวขนาดการจัดเรียงเซกชันในการกระจายตัวหน่วยความจำ (Block size) จำนวนเซกชันในหัวไฟล์ (Num sections)



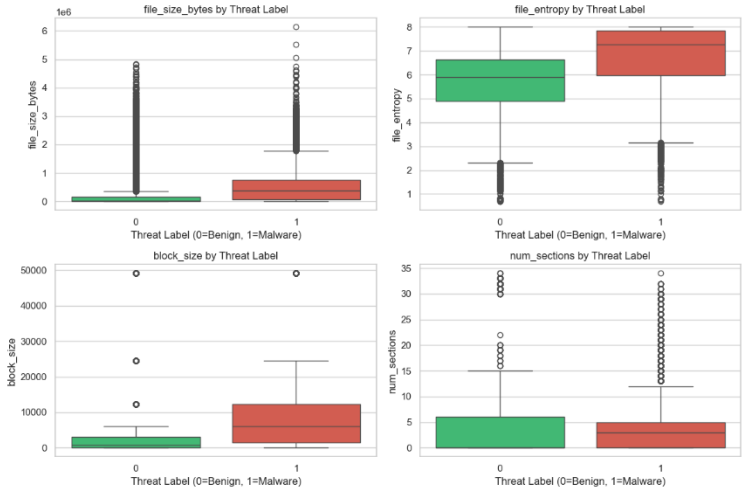
กราฟความหนาแน่นแสดงการกระจายตัวของข้อมูล 4 ตัวแปร

กราฟแท่งแสดงสัดส่วนของข้อมูลเป้าหมายระหว่างกลุ่มไฟล์ปกติ และกลุ่มมัลแวร์ ข้อมูลแสดงให้เห็นว่ามีกลุ่มไฟล์ปกติประมาณ 60,000 รายการ และกลุ่มมัลแวร์ประมาณ 45,000 รายการ



กราฟแท่งแสดงสัดส่วนของข้อมูลเป้าหมายระหว่างกลุ่มไฟล์ปกติ และกลุ่มมัลแวร์

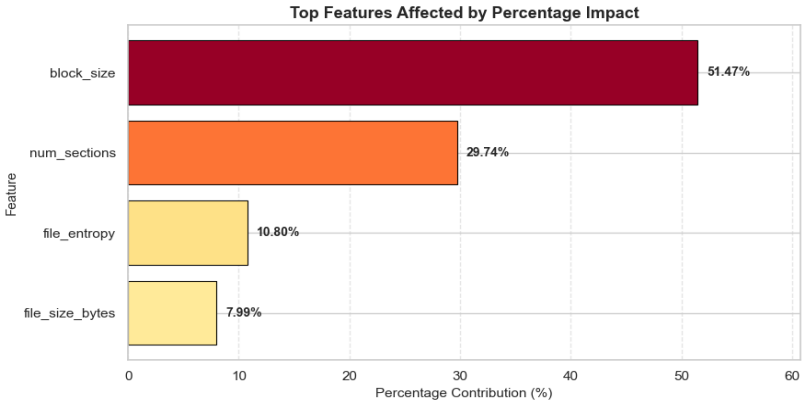
กราฟ Box Plot สำหรับแสดงการกระจายตัวเชิงลึกของ 4 ตัวแปรเดิม โดยแบ่งกลุ่มเปรียบเทียบตาม Threat Label (0 และ 1) กราฟนี้ใช้สำหรับดูค่ามัธยฐาน (Median), การกระจุกตัวของข้อมูล (Quartiles)



กราฟ Box Plot สำหรับแสดงการกระจายตัวเชิงลึกของ 4 ตัวแปร

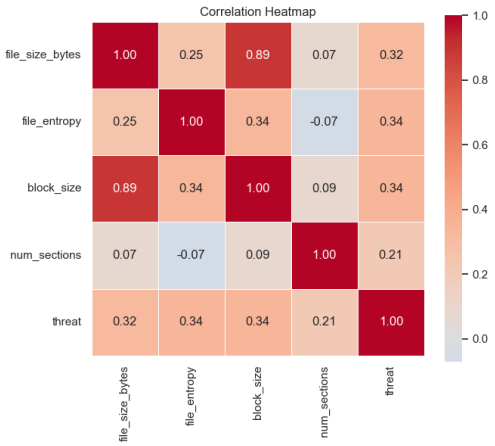
และจุดข้อมูลที่ผิดปกติ (Outliers) อย่างชัดเจน เช่น จะสังเกตเห็นได้ว่ากลุ่มมัลแวร์มีแนวโน้มของค่า เอนโทรปีที่สูงกว่ากลุ่มไฟล์ปกติ

กราฟแท่งแนวนอนที่สรุปและจัดอันดับตัวแปร (Features) ที่มีผลกระทบความสำคัญมากที่สุด



จัดอันดับตัวแปรที่มีผลกระทบความสำคัญมากที่สุด

แผนภาพความร้อน (Heatmap) แสดงค่าสัมประสิทธิ์สหสัมพันธ์ (Correlation Coefficient) เพื่อดูความสัมพันธ์ทางสถิติระหว่างตัวแปรแต่ละคู่ ค่าที่เข้าใกล้ 1.00 (สีแดงเข้ม) แสดงถึงความสัมพันธ์เชิงบวกที่สูงมาก เช่น ขนาดไฟล์ และ การจัดเรียงเซกชันในการกระจายตัวหน่วยความจำ มีความสัมพันธ์เกี่ยวข้องกันสูงถึง 0.89 กราฟนี้ยังแสดงให้เห็นว่าตัวแปรแต่ละตัวมีความสัมพันธ์กับตัวแปรเป้าหมาย (threat) อยู่ที่ระหว่าง 0.21 ถึง 0.34



แผนภาพความร้อนแสดงค่าสัมประสิทธิ์สหสัมพันธ์

ขั้นตอนที่ 5: การสอนปัญญาประดิษฐ์ (Model Training)

ระบบทำงานผ่านไฟล์ model_training.py

การเทรนโมเดล (Model Training) คือกระบวนการที่เราป้อนข้อมูลตัวอย่างให้คอมพิวเตอร์เรียนรู้รูปแบบและความสัมพันธ์ เพื่อให้มันสามารถตัดสินใจหรือทำนายข้อมูลใหม่ๆ ที่ไม่เคยเห็นมาก่อนได้ด้วยตัวเอง ในโปรเจกต์นี้คือการสอนให้ AI แยกแยะว่าไฟล์ไหนคือโปรแกรมปกติ และไฟล์ไหนคือมัลแวร์

นี่คือหลักการและขั้นตอนการเขียนโค้ดเพื่อเทรนโมเดลจากไฟล์ model_training.py เพื่อให้คุณเห็นภาพรวมและสามารถนำไปประยุกต์ใช้ได้

- **นำเข้าข้อมูลและกำหนดตัวแปร:** ดึงข้อมูลที่ผ่านการทำความสะอาดแล้วจากฐานข้อมูล SQLite (ตาราง malware_features_train_clean) จากนั้นกำหนดตัวแปรต้น (X) ที่โมเดลจะใช้ประกอบการตัดสินใจ ได้แก่ ขนาดไฟล์ เอนโทรปี ขนาดบล็อก และจำนวนเซกชัน และกำหนดตัวแปรเป้าหมาย (y) คือ threat ที่บอกว่าไฟล์นั้นปกติ (0) หรือเป็นมัลแวร์ (1)
- **แบ่งชุดข้อมูล (Train-Validation-Test Split):** เปรียบเสมือนการเตรียมข้อสอบให้ AI ระบบจะแบ่งข้อมูลออกเป็น 3 ส่วน คือ ชุดเรียนรู้ (Train 60%), ชุดสำหรับปรับจูนระหว่างเรียน (Validation 20%), และชุดข้อสอบปลายภาคที่ AI ไม่เคยเห็นมาก่อน (Test 20%)
- **สร้างโมเดล XGBoost:** โค้ดใช้อัลกอริทึม XGBClassifier ซึ่งทำงานคล้ายต้นไม้ตัดสินใจหลายๆ ต้นมาช่วยกันโหวต มีการตั้งค่าพารามิเตอร์สำคัญ เช่น ควบคุมความลึกของต้นไม้ (max_depth=5) และมีการใช้ scale_pos_weight เพื่อแก้ปัญหาในกรณีที่ข้อมูลไฟล์ปกติและมัลแวร์มีจำนวนไม่เท่ากัน (Imbalanced Data)
- **เริ่มกระบวนการสอน (Fit):** สั่งให้โมเดลเริ่มเรียนรู้จากชุด Train โดยใช้ชุด Validation เป็นตัวประเมิน ระบบมีกลไก early_stopping_rounds=30 ที่จะหยุดการเรียนรู้อัตโนมัติหากโมเดลไม่มีการพัฒนาขึ้นภายใน 30 รอบ เพื่อป้องกันไม่ให้ AI ท่องจำข้อมูล (Overfitting) มากเกินไป
- **หาจุดตัดที่เหมาะสม (Best Threshold):** โมเดลจะให้ผลลัพธ์เป็นค่าความน่าจะเป็น ระบบจึงต้องคำนวณหาจุดตัดคะแนน (Threshold) ที่ดีที่สุดจากชุด Validation เพื่อดูว่าควรใช้เกณฑ์กี่เปอร์เซ็นต์ในการฟันธงว่าเป็นมัลแวร์ โดยอิงจากค่า F1-Score ที่สูงที่สุด

- **ประเมินประสิทธิภาพ (Evaluate):** นำจุดตัดคะแนนที่ได้ มาใช้ทำนายผลกับชุด Test จากนั้นโค้ดจะคำนวณตัวชี้วัดต่างๆ เช่น ความแม่นยำ (Accuracy), ความสามารถในการจับมัลแวร์ (Recall), และแสดงผลเป็นตาราง Confusion Matrix กับกราฟเพื่อดูว่าโมเดลทายถูกหรือผิดในรูปแบบใดบ้าง
- **บันทึกพร้อมใช้งาน (Save Model):** เมื่อได้ผลลัพธ์ที่น่าพอใจ ระบบจะบันทึกโครงสร้างการตัดสินใจของ AI ออกมาเป็นไฟล์ malware_modelV2.bin เพื่อนำไปใช้งานกับระบบอื่นๆ ต่อไป

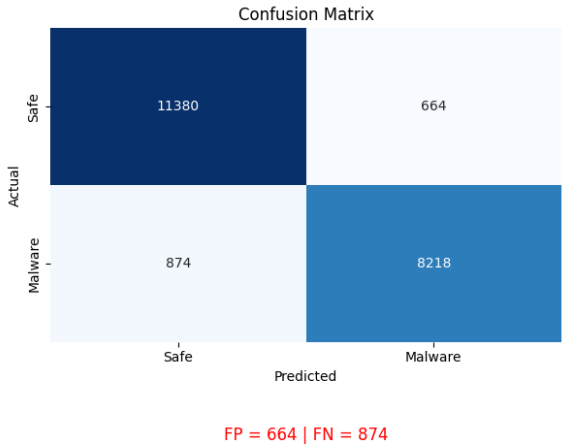
ขั้นตอนที่ 6: การประเมินประสิทธิภาพ (Evaluation)

หลังจากการสอนเสร็จสิ้น ไฟล์ evaluation.py จะทำหน้าที่ออกใบเกรดให้กับ AI

- **ทดสอบกับข้อสอบจริง:** ระบบจะดึงโมเดล malware_modelV2.bin ขึ้นมาทำนายผลกับชุดข้อมูล Test
- **วัดผลหลายมิติ:** โปรแกรมจะรายงานค่าความแม่นยำ (Accuracy), ความสามารถในการทายมัลแวร์ได้ถูกต้อง (Precision), และความสามารถในการจับมัลแวร์ไม่ให้หลุดรอด (Recall)
- **แสดงผลผ่านกราฟ:** ระบบจะแสดงกราฟ Confusion Matrix (ตารางแจกแจงการทายถูกผิด) และกราฟ ROC Curve เพื่อยืนยันว่า AI มีประสิทธิภาพในการแยกแยะจริงๆ

การประเมินประสิทธิภาพของโมเดลจะใช้กระบวนการทดสอบกับชุดข้อมูลทดสอบ (Testing Set) ที่โมเดลไม่เคยเห็นมาก่อน เพื่อวัดความสามารถในการทำงานร่วมกับโปรแกรม ssdeep-scanner โดยมีรายละเอียดและผลลัพธ์การประเมินดังนี้

4.1.1 Confusion Matrix ของ XGboost



ผลลัพธ์ของ **Confusion Matrix ของ XGboost**

ทดสอบด้วยข้อมูลรวม 21,136 ไฟล์ (แบ่งเป็นไฟล์ปลอดภัย 12,044 ไฟล์ และมัลแวร์ 9,092 ไฟล์) ทำนายถูกต้องว่าเป็นไฟล์ปลอดภัย 11,380 ไฟล์ (True Negative) ทำนายผิดพลาดว่าเป็นมัลแวร์ (False Positive / แจ้งเตือนภัยปลอม) 664 ไฟล์ ดักจับมัลแวร์ได้ถูกต้อง 8,218 ไฟล์ (True Positive) หลุดรอดการตรวจจับ (False Negative / มองมัลแวร์เป็นไฟล์ปลอดภัย) 874 ไฟล์

4.1.2 Classification Report ของ XGboost

	precision	recall	f1-score	support
0	0.9267	0.9462	0.9364	12044
1	0.9267	0.9267	0.9136	9092
accuracy			0.9267	21136

Classification Report ของ XGboost

กลุ่ม 0: ไฟล์ปกติ (Benign)

Precision: เมื่อโมเดลตัดสินว่า "เป็นไฟล์ปกติ" คำทำนายมีความถูกต้องถึง 92.67%

Recall: จากไฟล์ปกติที่มีอยู่จริงทั้งหมด 12,044 ไฟล์ โมเดลสามารถระบุตัวตนได้ถูกต้องถึง 94.62%

F1-Score: ค่าความสมดุลระหว่าง Precision และ Recall อยู่ในระดับสูงมาก สะท้อนถึงเสถียรภาพในการจัดการกับไฟล์ปกติ

กลุ่ม 1: มัลแวร์ (Malware)

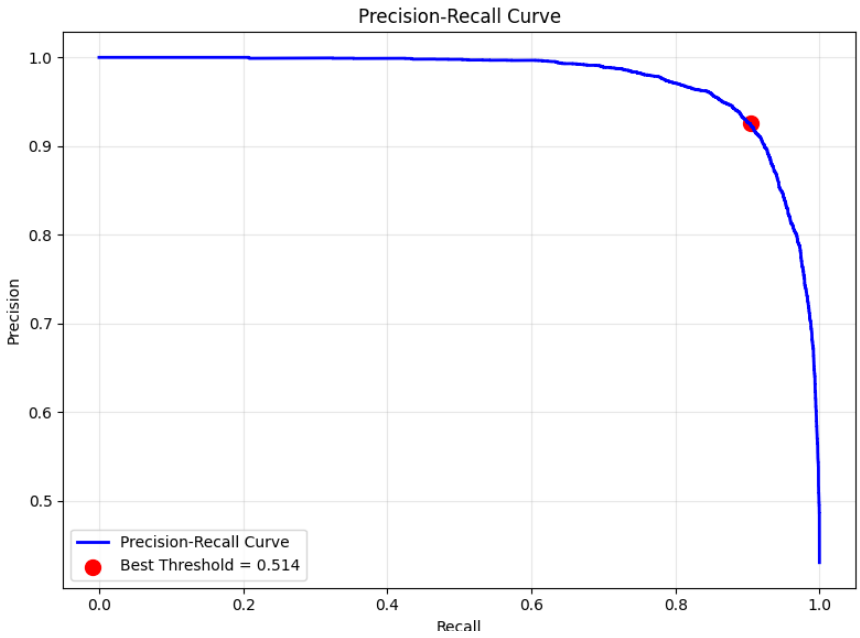
Precision: เมื่อโมเดลแจ้งเตือนว่า "เป็นมัลแวร์" มีความถูกต้อง 92.67% (มีอัตราการแจ้งเตือนผิดพลาด หรือ False Positive เพียง 7.33%)

Recall: จากมัลแวร์ที่มีอยู่จริง 9,092 ไฟล์ โมเดลสามารถดักจับได้ถึง 90.09% (มีมัลแวร์ที่หลุดรอด หรือ False Negative ประมาณ 9.91%)

F1-Score: คะแนนอยู่ในเกณฑ์ที่น่าพอใจ พิสูจน์ว่าโมเดลสามารถรักษามาตรฐานความแม่นยำได้ดีแม้ภัยคุกคามจะมีความซับซ้อน

Accuracy: จากการทดสอบกับไฟล์ทั้งหมดรวม 21,136 ไฟล์ แบบจำลองสามารถจัดหมวดหมู่ได้อย่างถูกต้องรวม 92.67%

4.1.3 Precision-Recall Curve (PR Curve) ของ XGboost

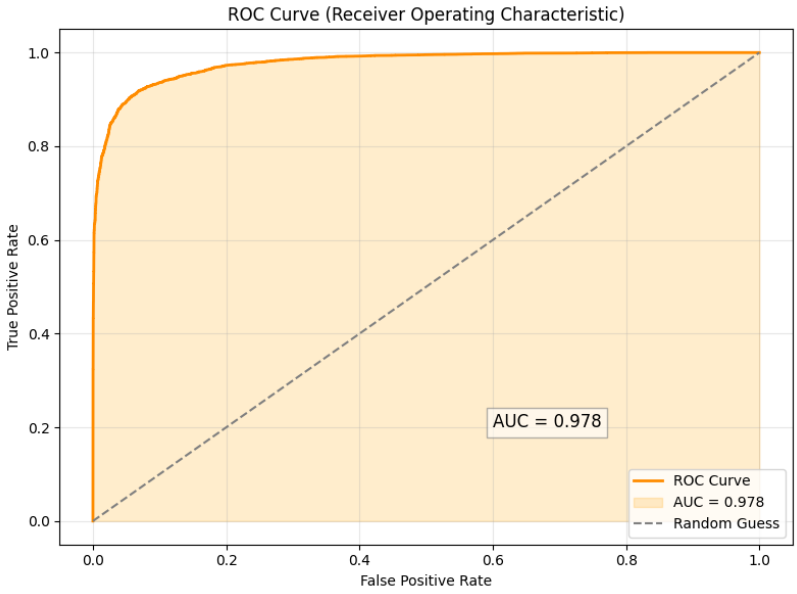


ผลลัพธ์ของ Precision-Recall Curve ของ XGboost

กราฟ Precision-Recall Curve แสดงความสัมพันธ์และการประนีประนอม (Trade-off) ระหว่างค่าความแม่นยำ (Precision) และค่าความครอบคลุม (Recall) ที่ระดับ Threshold ต่างๆ กราฟนี้เหมาะอย่างยิ่งสำหรับการประเมินประสิทธิภาพโมเดลในกรณีที่ชุดข้อมูลมีความไม่สมดุล (Class Imbalance)

- แกน X (Recall): สัดส่วนของมัลแวร์ทั้งหมดที่โมเดลตรวจจับได้
- แกน Y (Precision): สัดส่วนของการทำนายว่าเป็นมัลแวร์ที่ถูกต้อง
- จุดสีแดง (Best Threshold = 0.514): คือจุดบนเส้นโค้งที่ให้ค่า F1-Score สูงที่สุด

4.1.4 ROC-AUC Curve ของ XGboost



ผลลัพธ์ของ ROC-AUC Curve ของ XGboost

กราฟ ROC Curve แสดงประสิทธิภาพของโมเดลในการแยกแยะระหว่างคลาสปกติและมัลแวร์ในทุกๆ ระดับ Threshold โดยเทียบอัตราการพบผลบวกจริง (True Positive Rate) กับอัตราการพบผลบวกเทียม (False Positive Rate)

- แกน X (False Positive Rate - FPR): อัตราที่โมเดลทายผิดว่าไฟล์ปกติเป็นมัลแวร์
- แกน Y (True Positive Rate - TPR / Recall): อัตราที่โมเดลทายมัลแวร์ได้ถูกต้อง
- เส้นประสีเทา (Random Guess): เส้นฐานที่แสดงประสิทธิภาพของการเดาสุ่ม (AUC = 0.5)
- พื้นที่ใต้กราฟ (AUC = 0.978): ค่า Area Under the Curve (AUC) ที่ 0.978 (เข้าใกล้ 1.0) แสดงให้เห็นว่าโมเดลมีความสามารถดีเยี่ยมในการจัดลำดับความน่าจะเป็น และสามารถแยกแยะมัลแวร์ออกจากไฟล์ปกติได้อย่างมีประสิทธิภาพสูงมาก

สรุป

หากต้องการใช้งานโค้ดชุดนี้ เริ่มต้นจากการหาโฟลเดอร์ที่มีโปรแกรมปกติ และโฟลเดอร์ที่เก็บมัลแวร์ จากนั้นรัน main.py แล้วเลือกเมนู **Data Gathering** เพื่อเก็บข้อมูลทีละโฟลเดอร์ เมื่อได้ฐานข้อมูลที่สมบูรณ์แล้ว ให้เข้าเมนู **EDA/Model Training** เพื่อคัดกรองข้อมูลและสอนโมเดล ในท้ายที่สุดคุณจะได้ไฟล์ .bin ซึ่งสามารถนำไปใช้เขียนโปรแกรมดักจับมัลแวร์ในระบบอื่นๆ ต่อไปได้ทันที